Polymarket Reward Farming Agent — Full System Specification

1. Objective

Build an autonomous agent that maximizes net profit from Polymarket reward farming using data-driven decision making under uncertainty.

2. Core EV Model

```
EV = (reward_rate * expected_time_on_book)
     - (P_fill * E_loss_given_fill)
```

This is the central decision equation used for all trade placement.

3. Key Models

3.1 Fill Probability Model

```
p_fill = model.predict(features)
```

3.2 Loss Model (conditional)

```
loss = loss_model.predict(features + [p_fill])
```

3.3 Hazard Model

```
hazard = hazard_model.predict(features)
expected_time = 1 / hazard
```

4. Reward Attribution System

Since per-market rewards are not available via API, we reconstruct them.

```
contribution_i = sum(q_score * time)
```

```
reward_i = total_reward * (contribution_i / total_contribution)
```

Apply smoothing:

```
reward_i = 0.7 * prev + 0.3 * new
```

5. Data Architecture

```
book_snapshots(ts, market, spread, depth, imbalance)
fills(market, price, slippage, ts)
```

```
orders(market, placed_ts, cancel_ts)
reward_daily(date, total_reward)
market_rewards_estimated(date, market, reward)
```

6. Feature Set

```
features = {
    spread,
    depth,
    imbalance,
    volatility,
    q_score,
    time_of_day,
    market_category
}
```

7. Decision Loop

```
for market in markets:
    features = extract(market)

    p_fill = fill_model.predict(features)
    loss = loss_model.predict(features + [p_fill])
    hazard = hazard_model.predict(features)

    expected_time = 1 / hazard
    reward_rate = reward_model.predict(market)

    ev = (reward_rate * expected_time) - (p_fill * loss)

    if ev > threshold:
        place_order(market)
```

8. Risk Management

```
if drawdown > 15%:
    stop_trading()

if hourly_loss > threshold:
    reduce_exposure()

if ev < min_threshold:
    skip_market()
```

9. Capital Allocation

```
capital_scale = median_ev / historical_median_ev
budget = total_capital * capital_scale
```

10. Learning Loop

```
daily:
    attribute_rewards()
    retrain_models()
    update_bandit()
```

11. Market Selection (Bandit)

```
sample = Beta(alpha, beta)
select highest sampled markets
```

12. Scraper Integration

```
run playwright scraper
fetch per-market rewards
validate attribution model
```

13. Deployment Phases

Phase 0: Data logging

Phase 1: Shadow models

Phase 2: A/B testing

Phase 3: Partial automation

Phase 4: Full autonomy

14. Key Principles

1. Do not force trades
2. Avoid high-toxicity fills
3. Adapt faster than competitors
4. Use real reward feedback